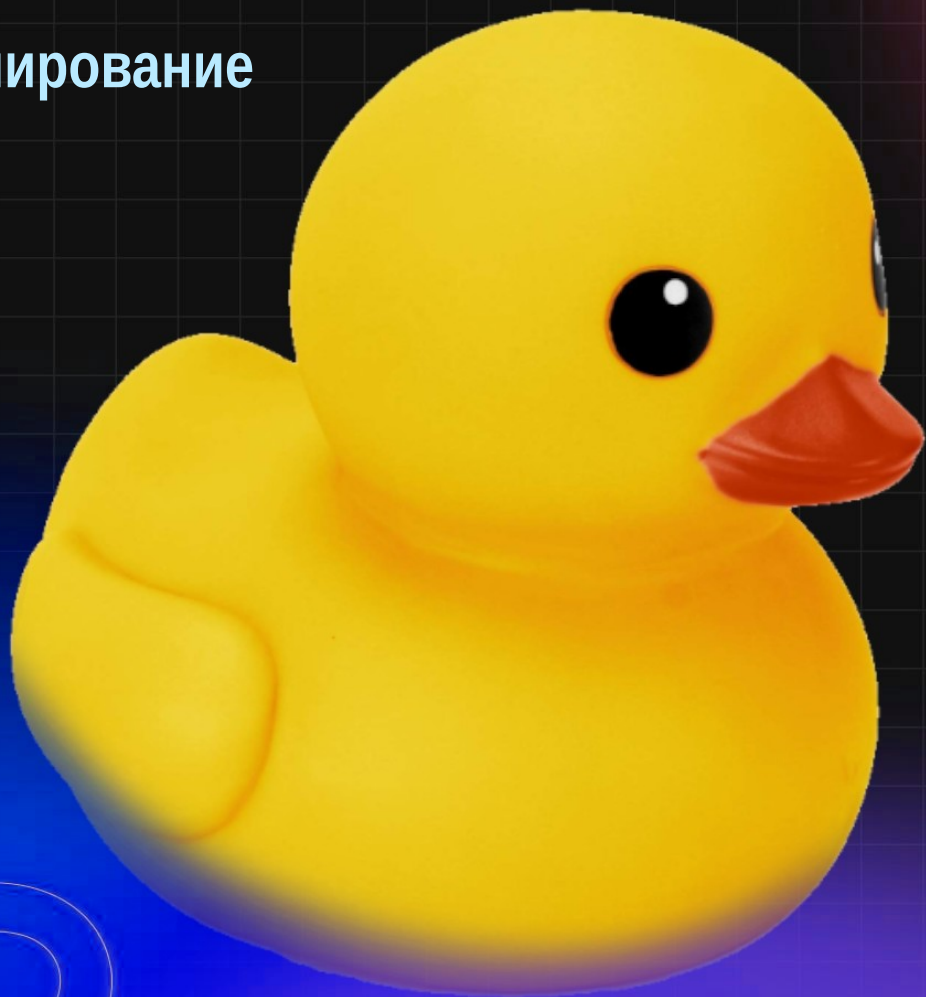


Программирование
2 семестр
2026



ІТМО

Введение

Гаврилов Антон Валерьевич

- лабы 5, 6, 7 (за лабу 12...15, в Барс - $12+24 = 36$)
 - ❖ все, что больше 36 - в резерве (0...9)
- рубеж (12...20)

допуск до экзамена

автомат 3Е, если лабы+резерв+рубеж ≥ 60

- лаба 8 (12...15) + резерв от лаб 5-7 = (12...24)
 - ❖ **автомат 5А**, если > 70 после лаб и рубежки
- экзамен (12...20) [+ 3 доп. балла]

- лабы 5, 6, 7 (за лабу 9...15, в Барс - $9+9+9+9 = 36$)
 - ❖ все, что больше 36 - в резерве (0,,,9)
- рубеж (12...20)

допуск до экзамена

автомат 3Е, если лабы+резерв+рубеж ≥ 60

- лаба 8 (9...15) + резерв от лаб 5-7 = (9...24)
 - ❖ **автомат 5А**, если > 70 после лаб и рубежки
- экзамен (12...20) [+ 3 доп. балла]

- Лабы (КТ) (48 - 72)

- ❖ 3 лабы * 2 = 6 этапов
- ❖ 2-4 балла за код
- ❖ 4-8 за понимание кода
- ❖ 8-12 баллов за этап

- Теория (КТ) (6 - 10)

- ❖ защита на лабах
- ❖ рубежка
- ❖ экзамен

- Мини-задания (0 - 6)

- ❖ 0-1 балл за этап лабы

- Доп. задание (9-15)

- ❖ на повышение оценки

● Сценарий 1

- 48-54 - лабы
- 6-8-10 - теория
- 6 - мини-задания

● Сценарий 2

- 54-60 - лабы
- 6-8-10 - теория
- 0-6 - мини-задания

● Сценарий 3

- 60-72 - лабы
- 6-8-**10** - теория
- 0-6 - мини-задания
- 9-15 - доп.задание

● Итого - 60-70
(3E-3D)

● Итого - 60-76
(3E - 4C)

● Итого - 75-103
(4C-5A)

- Дженирики и Коллекции
- Ввод-вывод
- Функциональное программирование
- Сетевое взаимодействие (кроме Инфохимии)
- Многопоточность
- Работа с базами данных
- Шаблоны проектирования
- Графический интерфейс и локализация

- <https://se.ifmo.ru>

- ❖ задания к лабораторным
- ❖ видео и тексты лекций
- ❖ методички
- ❖ вопросы к экзамену



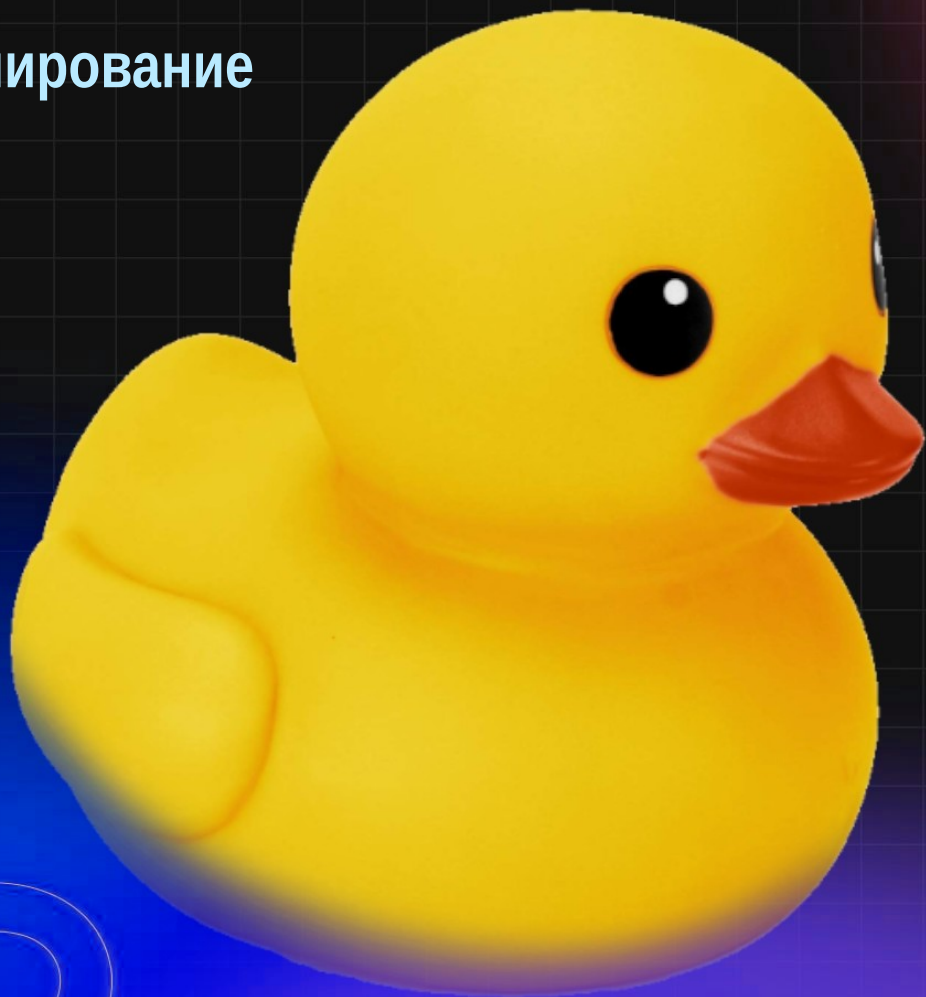
- <https://docs.oracle.com/javase/8/docs/api/>

- <https://docs.oracle.com/en/java/javase/17/docs/api/>

- <https://docs.oracle.com/en/java/javase/21/docs/api/>

Программирование
2 семестр
2026

ІТМО



Типы данных
Обертки
(Wrappers)

- Тип данных
 - ❖ диапазон значений + операции
- Типобезопасность
 - ❖ Проверка совместимости типа
- Статическая и динамическая типизация
 - ❖ момент связывания переменной с типом
- Сильная и слабая типизация
 - ❖ явное и неявное приведение

- Статическая типизация
 - ❖ тип переменной при объявлении
 - ❖ компилятор контролирует типы
 - ❖ оптимизация на уровне машинных кодов
- Динамическая типизация
 - ❖ тип есть у значения, у переменной - при присваивании
 - ❖ код проще и более гибкий
 - ❖ больше ошибок

- Сильная
 - ❖ минимум неявного приведения
 - ❖ запрет операций над несовместимыми типами
- Слабая
 - ❖ неявное приведение
 - ❖ операции с разными типами
 - ❖ JavaScript
 - <https://www.destroyallsoftware.com/talks/wat>

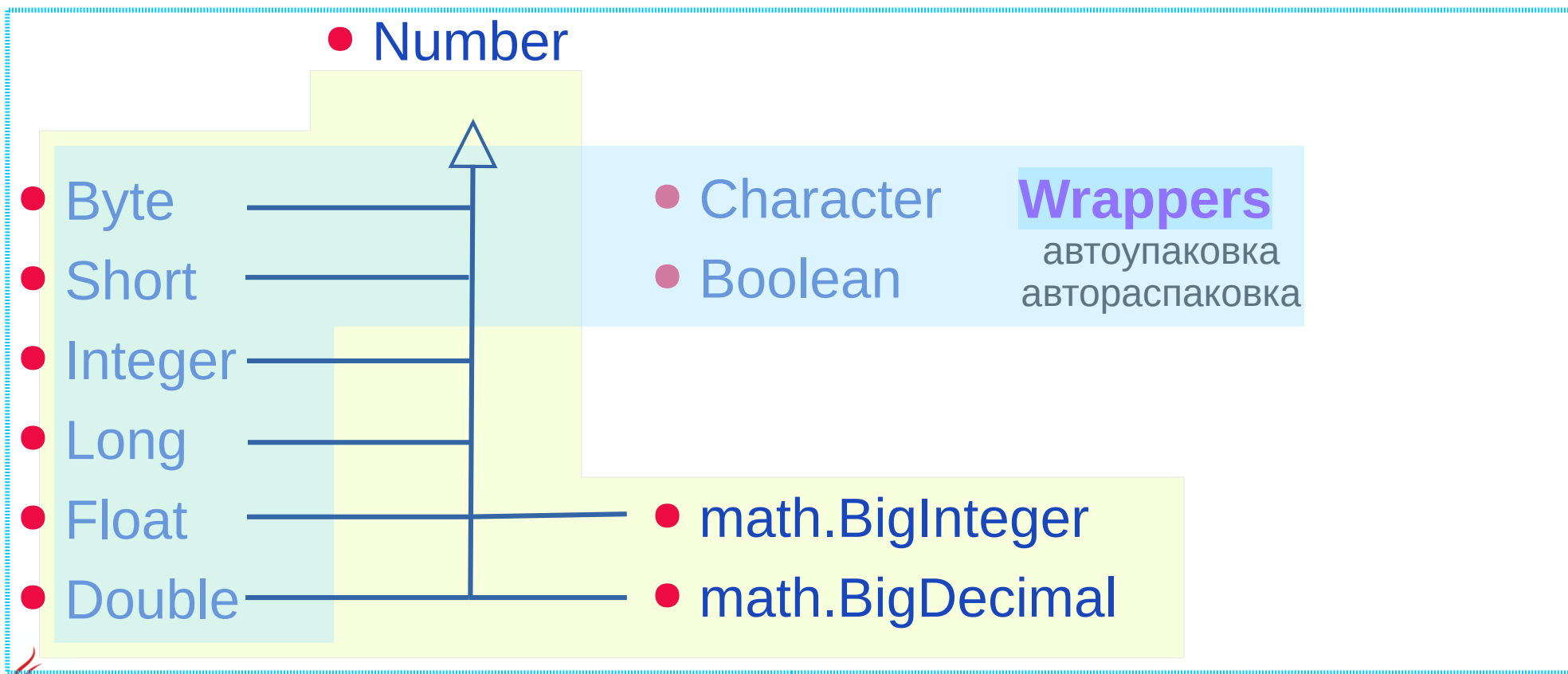
(C) Gary Bernhardt, 2012

- Примитивные

- ❖ значение
- ❖ не наследник Object
- ❖ эффективные
- ❖ операции
- ❖ массивы
- ❖ не обобщается

- Ссылочные

- ❖ ссылка
- ❖ наследник Object
- ❖ больше и медленнее
- ❖ методы
- ❖ массивы и коллекции
- ❖ обобщается



- abstract class

- методы

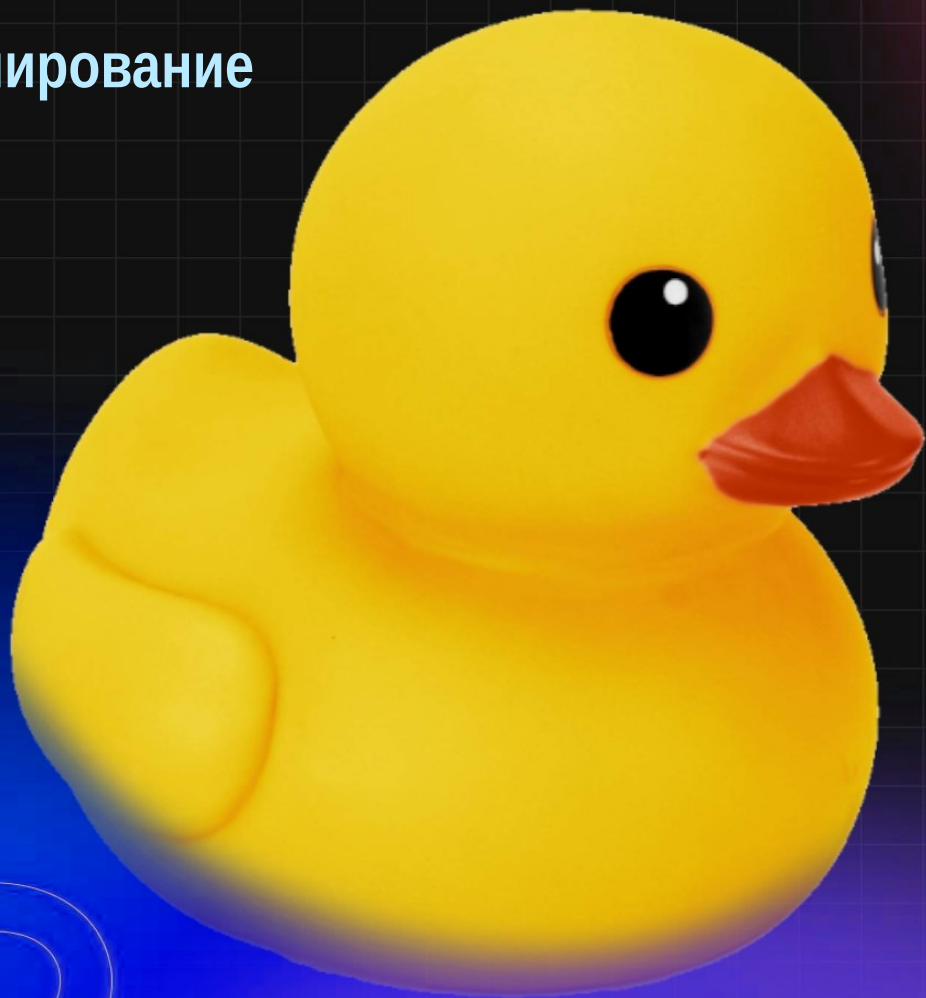
- ❖ byte byteValue()
- ❖ short shortValue()
- ❖ int intValue()
- ❖ long longValue()
- ❖ float floatValue()
- ❖ double doubleValue()

- ПОТОМКИ

- ❖ Byte
- ❖ Short
- ❖ Integer
- ❖ Long
- ❖ BigInteger
- ❖ Float
- ❖ Double
- ❖ BigDecimal

- 

Программирование
2 семестр
2026



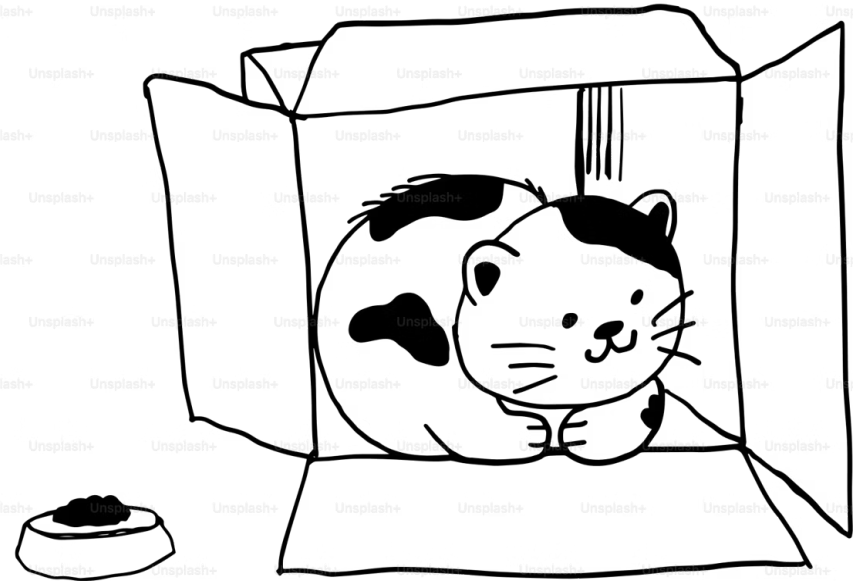
ІТМО

Обобщения
Generics

- Обобщенное программирование
 - ❖ абстрактное описание данных и алгоритмов, применяемое без изменения к разным типам данных
- Параметрический полиморфизм
 - ❖ тип данных является параметром
- Дополнительный контроль на этапе компиляции
 - ❖ более безопасный код

- Перегрузка методов (overriding)
 - ❖ разный код для разных типов данных
 - ❖ связывание на этапе компиляции
- Полиморфизм подтипов (ООП, overloading)
 - ❖ перегруженный код для наследуемых типов
 - ❖ связывание на этапе выполнения
- Параметрический полиморфизм
 - ❖ разные типы, один код
 - ❖ тип является параметром

```
public class Box {  
  
    private Object obj;  
  
    void put(Object o) {  
        obj = o;  
    }  
  
    Object get() {  
        return obj;  
    }  
}
```



Picture by [Runend Art](#) on [Unsplash](#)

Strging -> Box -> String ?

```
public class Box {  
    private Object obj;  
  
    void put(Object o) {  
        obj = o;  
    }  
  
    Object get() {  
        return obj;  
    }  
}
```

```
Box box = new Box();  
box.put("Hello");  
  
var obj = box.get();
```

Strging -> Box -> String ?

```
public class Box {  
    private Object obj;  
  
    void put(Object o) {  
        obj = o;  
    }  
  
    Object get() {  
        return obj;  
    }  
}
```

```
Box box = new Box();  
box.put("Hello");  
  
var obj = box.get();  
  
int len = obj.length();
```

Stringing -> Box -> String ?

```
public class Box {  
    private Object obj;  
  
    void put(Object o) {  
        obj = o;  
    }  
  
    Object get() {  
        return obj;  
    }  
}
```

```
Box box = new Box();  
box.put("Hello");  
  
var obj = box.get();  
  
int len = obj.length();  
  
error: cannot find symbol  
symbol:   method length()  
location: class Object
```

Stringing -> Box -> String ?

```
public class Box {  
    private Object obj;  
  
    void put(Object o) {  
        obj = o;  
    }  
  
    Object get() {  
        return obj;  
    }  
}
```

```
Box box = new Box();  
box.put("Hello");  
  
String obj = box.get();  
  
int len = obj.length();
```

Stringing -> Box -> String ?

```
public class Box {  
    private Object obj;  
  
    void put(Object o) {  
        obj = o;  
    }  
  
    Object get() {  
        return obj;  
    }  
}
```

```
Box box = new Box();  
box.put("Hello");  
  
String obj = box.get();  
  
int len = obj.length();  
  
error: incompatible types: Object  
cannot be converted to String  
        String y = box.get();
```


Stringing -> Box -> String ?

```
public class Box {  
    private Object obj;  
  
    void put(Object o) {  
        obj = o;  
    }  
  
    Object get() {  
        return obj;  
    }  
}
```

```
Box box = new Box();  
box.put("Hello");  
  
String obj = (String) box.get();  
  
int len = obj.length();  
  
// компиляция без ошибок!
```

Integer -> Box -> String ?

```
public class Box {  
    private Object obj;  
  
    void put(Object o) {  
        obj = o;  
    }  
  
    Object get() {  
        return obj;  
    }  
}
```

```
Box box = new Box();  
box.put(256);  
  
String obj = (String) box.get();  
int len = obj.length();  
  
// компиляция без ошибок!
```

Integer -> Box -> String ?

```
public class Box {  
    private Object obj;  
  
    void put(Object o) {  
        obj = o;  
    }  
  
    Object get() {  
        return obj;  
    }  
}
```

```
Box box = new Box();  
box.put(256);  
  
String obj = (String) box.get();  
  
int len = obj.length();  
  
Exception in thread "main"  
java.lang.ClassCastException:  
class java.lang.Integer cannot be cast  
to class java.lang.String  
  
// ошибка при выполнении!!
```

```
public class Box {  
    private Object obj;  
  
    void put(Object o) {  
        obj = o;  
    }  
  
    Object get() {  
        return obj;  
    }  
}
```

```
Box box = new Box();  
box.put(256);  
int len;  
Object obj = box.get();  
if (obj instanceof String) {  
    len = obj.length();  
} else {  
    len = -1;  
}
```

```
public class Box {  
  
    private Object obj;  
  
    void put(Object o) {  
        obj = o;  
    }  
  
    Object get() {  
        return obj;  
    }  
}
```

```
public class Box<T> {  
  
    private T obj;  
  
    void put(T o) {  
        obj = o;  
    }  
  
    T get() {  
        return obj;  
    }  
}
```

```
var box= new Box<String>();  
box.put("Hello");  
  
String obj = box.get();  
  
int len = obj.length();
```

Box<T> T - параметр

Box<String> T = String

```
public class Box<T> {  
  
    private T obj;  
  
    void put(T o) {  
        obj = o;  
    }  
  
    T get() {  
        return obj;  
    }  
}
```

```
var box= new Box<String>();  
box.put(256);  
  
String obj = box.get();  
  
int len = obj.length();  
  
// ошибка при компиляции!  
  
error: incompatible types: Integer  
cannot be converted to String
```

```
public class Box<T> {  
  
    private T obj;  
  
    void put(T o) {  
        obj = o;  
    }  
  
    T get() {  
        return obj;  
    }  
}
```

```
public class Box<T>
```

Обобщенный тип
(generic)

```
Box<String> box = new Box<String>()  
var box = new Box<String>  
Box<String> box = new Box<>()
```

Параметризованный тип
(parametrized)

Diamond operator


```
class Pair<T, U> {  
    T obj1;  
    U obj2;  
}  
  
var p = new Pair<Integer, String>()  
  
var p2 = new Pair<Integer, Box<String>>()  
  
var p3 = new Pair<Pair<Integer, String>, Box<Box<String>>>()
```

- E — элемент коллекции (в Java API)
- K — ключ
- V — значение
- N — число
- T — первый параметр типа
- U, V, S — второй, третий, четвертый параметр типа

- C++ - templates
 - ❖ **разный код** для каждого параметра
 - ❖ во время исполнения **недоступна** информация о шаблонах
- C# - generics
 - ❖ **разный код** для каждого примитивного параметра
 - ❖ во время исполнения **доступна** информация об обобщениях
- Java - generics
 - ❖ при компиляции создается **код без обобщений (type erasure)**
 - ❖ во время исполнения **недоступна** информация об обобщениях

- В байткоде все обобщения стираются - type erasure
- Остаются только базовые типы - raw types
- Компилятор создает мостовые методы - bridge methods

```
class Box<T> { }  
  
class BBox extends Box<Byte> {  
    Byte get() {  
        return (Byte) super.get();  
    }  
}
```

```
class BBox extends Box<Byte> {  
    Byte get() {  
        return (Byte) super.get();  
    }  
    Object get() {  
        return get();  
    }  
}
```

- В байткоде все обобщения стираются - type erasure
- Остаются только базовые типы - raw types
- Компилятор создает мостовые методы - bridge methods

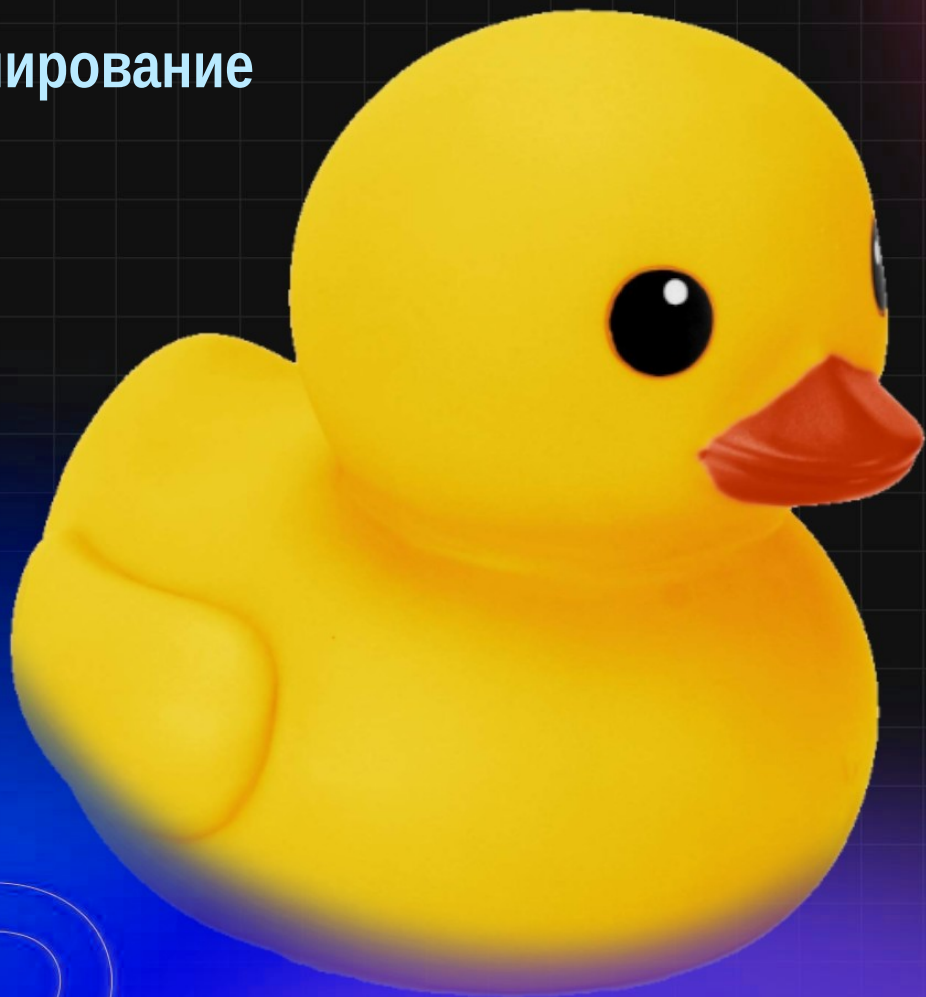
```
class Box<T> {  
    void put(T x) { ... }  
}  
class BBox extends Box<Byte> {  
    void put(Byte b) {  
        super.put(b);  
    }  
}
```

```
class Box {  
    void put(Object x) { ... }  
}  
class BBox extends Box {  
    void put(Byte b) {  
        super.put(b);  
    }  
    void put(Object x) {  
        put((Byte)x);  
    }  
}
```

- Совместимость со старыми версиями
- Недостаточная типобезопасность
 - ❖ Контроль только при компиляции
 - ❖ При выполнении возможны нарушения
- Нельзя:
 - ❖ Использовать generic примитив
 - `Box<int>`
 - ❖ Создавать объект параметра типа
 - `E e = new E()`
 - ❖ Создавать массивы обобщений
 - `Box<Cat>[]`
 - ❖ Перегружать методы с обобщениями
 - `void print(Box<Integer>)`
 - `void print(Box<String>)`

Программирование
2 семестр
2026

ІТМО

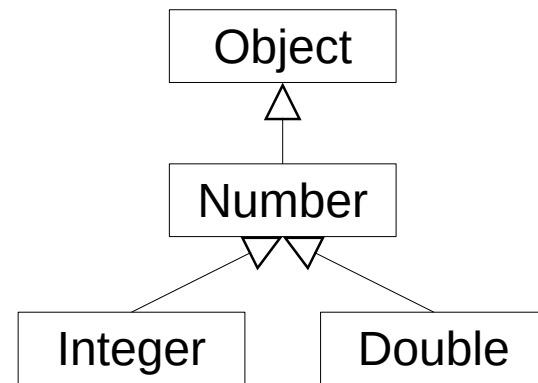


Обобщенные
методы

- Параметр типа перед типом возвращаемого значения
`public static <T> Box<T> pack(T obj)`
- При вызове метода нужен явный или неявный параметр
`Box<Integer> b1 = Box.<Integer>pack(1);`
`Box<String> b2 = Box.pack("Hello");`

- Box<T>

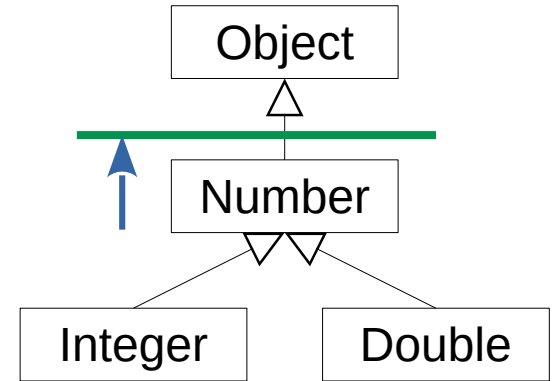
- ❖ T - любой тип (потомок Object)

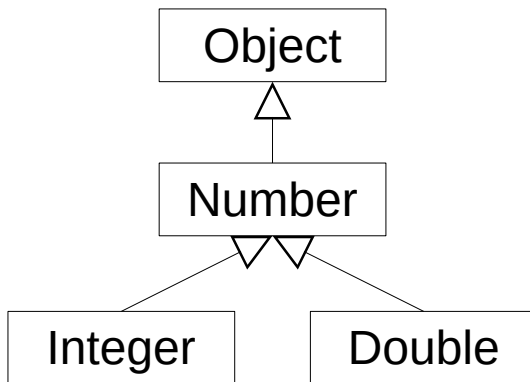


```
public class NumBox<T> {  
    T obj;  
  
    public int plusOne() {  
        return obj + 1;  
    }  
}
```

- Сложение только для чисел

```
public class NumBox<N extends Number> {  
    N num;  
  
    public int plusOne() {  
        return num.intValue() + 1;  
    }  
}
```

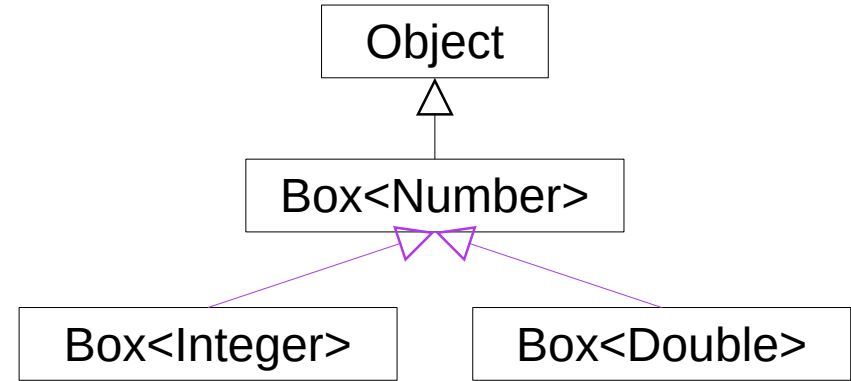
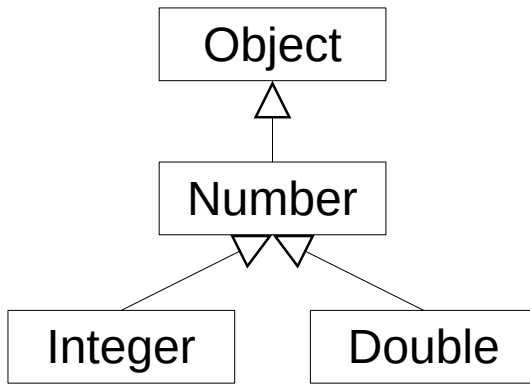




```
Number num;
```

```
Integer i = 42;  
num = i;
```

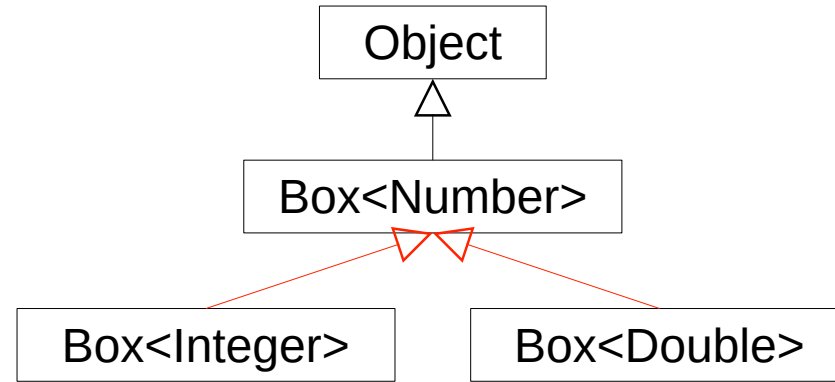
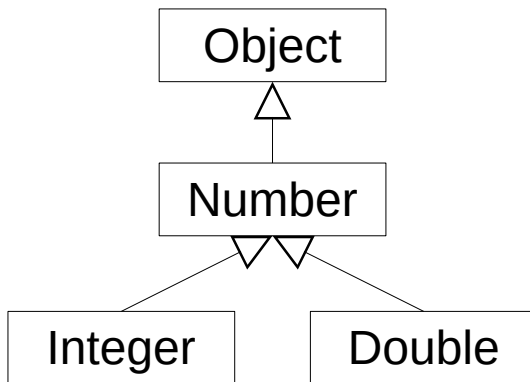
```
Double d = 3.14;  
num = d;
```



```
Box<Number> bnum = new Box<>();
```

```
Integer i = 42;  
bnum.put(i);
```

```
Double d = 3.14;  
bnum.put(d);
```

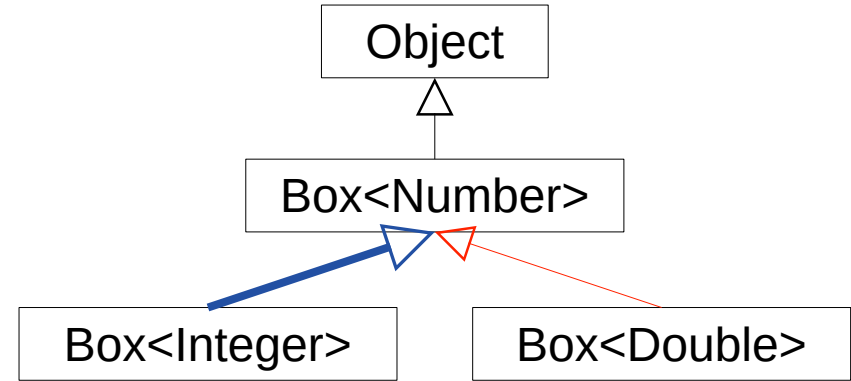
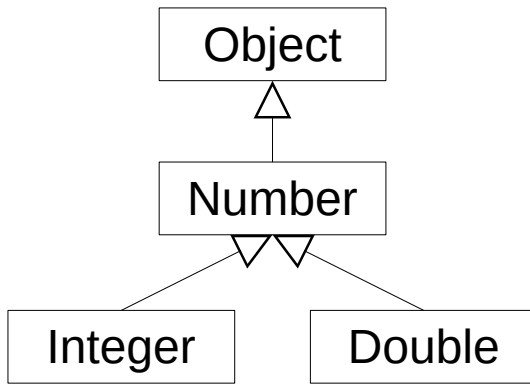


```
Box<Integer> bint = new Box<>();  
Box<Number> bnum = new Box<>();
```

```
Integer i = 42;  
bint.put(i);  
bint.put(d);
```

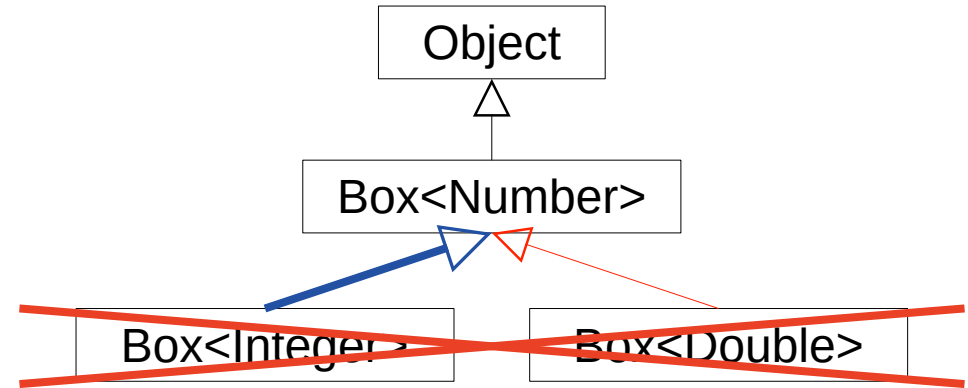
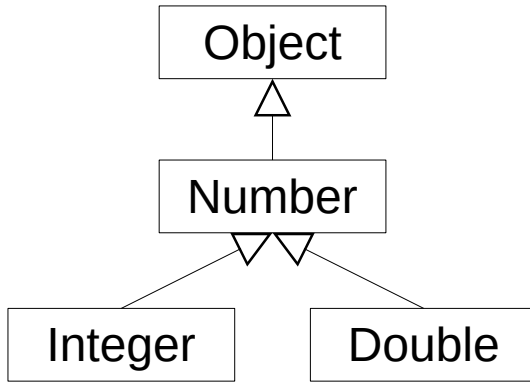
```
Double d = 3.14;
```

```
// Double cannot be converted to Integer
```

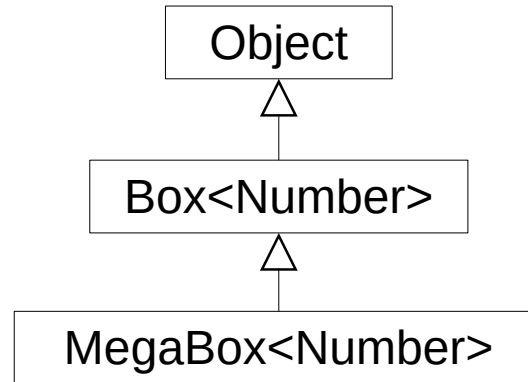
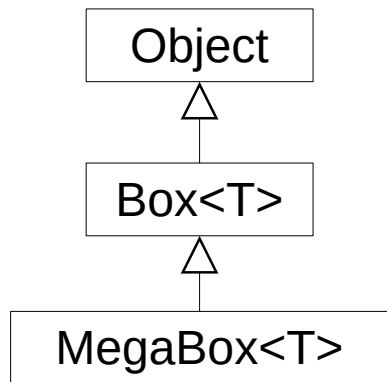


```
Box<Integer> bint = new Box<>();  
Box<Number> bnum = (Box<Number>) bint;
```

```
Integer i = 42;           Double d = 3.14;  
bnum.put(i);  
bnum.put(d);
```



```
Box<Integer> bint = new Box<>();  
Box<Number> bnum = (Box<Number>) bint;  
// Box<Integer> cannot be converted to Box<Number>  
Integer i = 42;      Double d = 3.14;  
bnum.put(i);  
bnum.put(d);
```



```
MegaBox<Number> mnum = new MegaBox<>();  
Box<Number> bnum = (Box<Number>) mnum;
```

```
bnum.put(2);  
System.out.println(bnum.get()) // 2000000
```



```
public class BoxAdder<N extends Number> {  
  
    public double sum(Box<N> b1, Box<N> b2) {  
        return b1.get().doubleValue() + b2.get().doubleValue();  
    }  
}
```

```
BoxAdder<Number> = new BoxAdder<>();  
Box<Number> box1 = new Box<>();  
Box<Number> box2 = new Box<>();  
box1.put(42);  
box2.put(3.14);  
  
System.out.println(BoxAdder.sum(box1, box2));
```

```
public class BoxAdder<N extends Number> {  
  
    public double sum(Box<N> b1, Box<N> b2) {  
        return b1.get().doubleValue() +  
        b2.get().doubleValue();  
    }  
}
```

```
BoxAdder<Number> = new BoxAdder<>();  
Box<Integer> box1 = new Box<>();  
Box<Double> box2 = new Box<>();  
box1.put(42);  
box2.put(3.14);
```

```
System.out.println(BoxAdder.sum(box1, box2));  
// Box<Integer> cannot be converted to Box<Number>  
// Box<Double> cannot be converted to Box<Number>
```

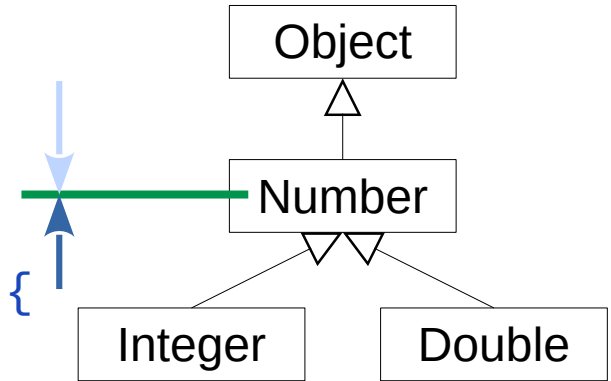
```
public class BoxAdder<N extends Number> {  
  
    public double sum(Box<? extends Number> b1, Box<? extends Number> b2) {  
        return b1.get().doubleValue() + b2.get().doubleValue();  
    }  
}
```

? - wildcard



```
BoxAdder<Number> = new BoxAdder<>();  
Box<Number> box1 = new Box<>();  
Box<Number> box2 = new Box<>();  
box1.put(42);  
box2.put(3.14);  
  
System.out.println(BoxAdder.sum(box1, box2));
```

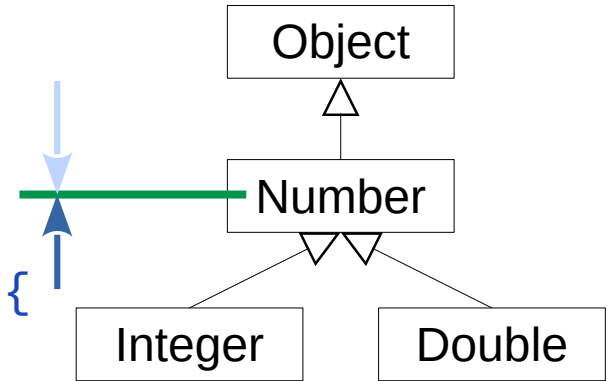
```
public class Box<T> {  
    private T obj;  
  
    public void copyTo(Box<? super T> to) {  
        to.obj = obj;  
    }  
    public void copyFrom(Box<? extends T> from) {  
        obj = from.obj;  
    }  
}
```



```
Box<Number> nbox = new Box<>();  
Box<Integer> ibox = new Box<>(); ibox.put(42);  
Box<Object> obox = new Box<>();  
nbox.copyFrom(ibox); // T = Number, ? = Integer  
nbox.copyTo(obox);    // T = Number, ? = Object  
ibox.copyTo(nbox);
```

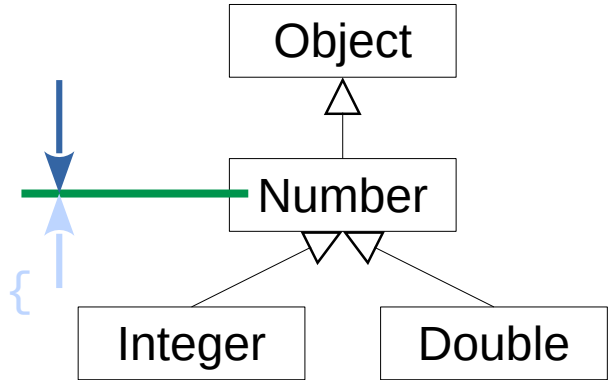
```
public class Box<T> {  
    private T obj;  
  
    public void copyTo(Box<? super T> to) {  
        to.obj = obj;  
    }  
    public void copyFrom(Box<? extends T> from) {  
        Number obj = from.obj;  
    }  
}
```

Number, Integer, Double



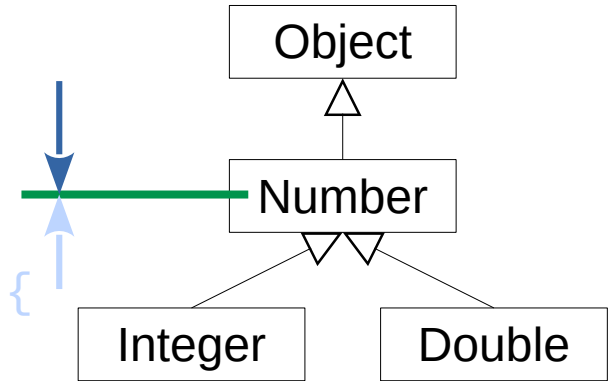
```
Box<Number> nbox = new Box<>();  
Box<Integer> ibox = new Box<>(); ibox.put(42);  
Box<Object> obox = new Box<>();  
nbox.copyFrom(ibox); // T = Number, ? = Integer  
nbox.copyTo(obox);    // T = Number, ? = Object  
ibox.copyTo(nbox);
```

```
public class Box<T> {  
    private T obj;  
  
    public void copyTo(Box<? super T> to) {  
        to.obj = obj;  
    }  
    public void copyFrom(Box<? extends T> from) {  
        obj = from.obj;  
    }  
}
```



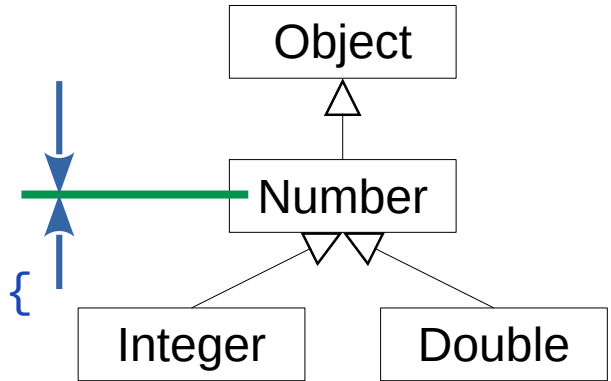
```
Box<Number> nbox = new Box<>();  
Box<Integer> ibox = new Box<>(); ibox.put(42);  
Box<Object> obox = new Box<>();  
nbox.copyFrom(ibox); // T = Number, ? = Integer  
nbox.copyTo(obox);    // T = Number, ? = Object  
ibox.copyTo(nbox);
```

```
public class Box<T> {  
    private T obj;  
  
    public void copyTo(Box<? super T> to) {  
        Object, to.obj = obj;  
        Number           Number  
    }  
    public void copyFrom(Box<? extends T> from) {  
        obj = from.obj;  
    }  
}
```



```
Box<Number> nbox = new Box<>();  
Box<Integer> ibox = new Box<>(); ibox.put(42);  
Box<Object> obox = new Box<>();  
nbox.copyFrom(ibox); // T = Number, ? = Integer  
nbox.copyTo(obox);    // T = Number, ? = Object  
ibox.copyTo(nbox);
```

```
public class Box<T> {  
    private T obj;  
  
    public void copyTo(Box<? super T> to) {  
        to.obj = obj;  
    }  
    public void copyFrom(Box<? extends T> from) {  
        obj = from.obj;  
    }  
}
```



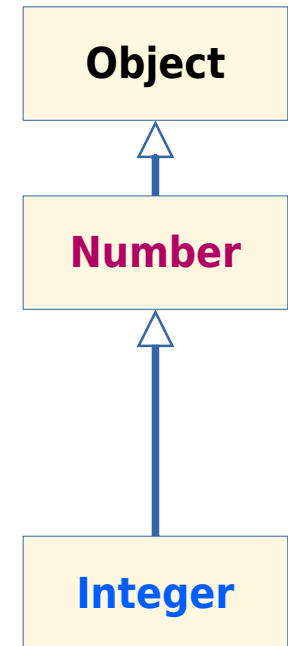
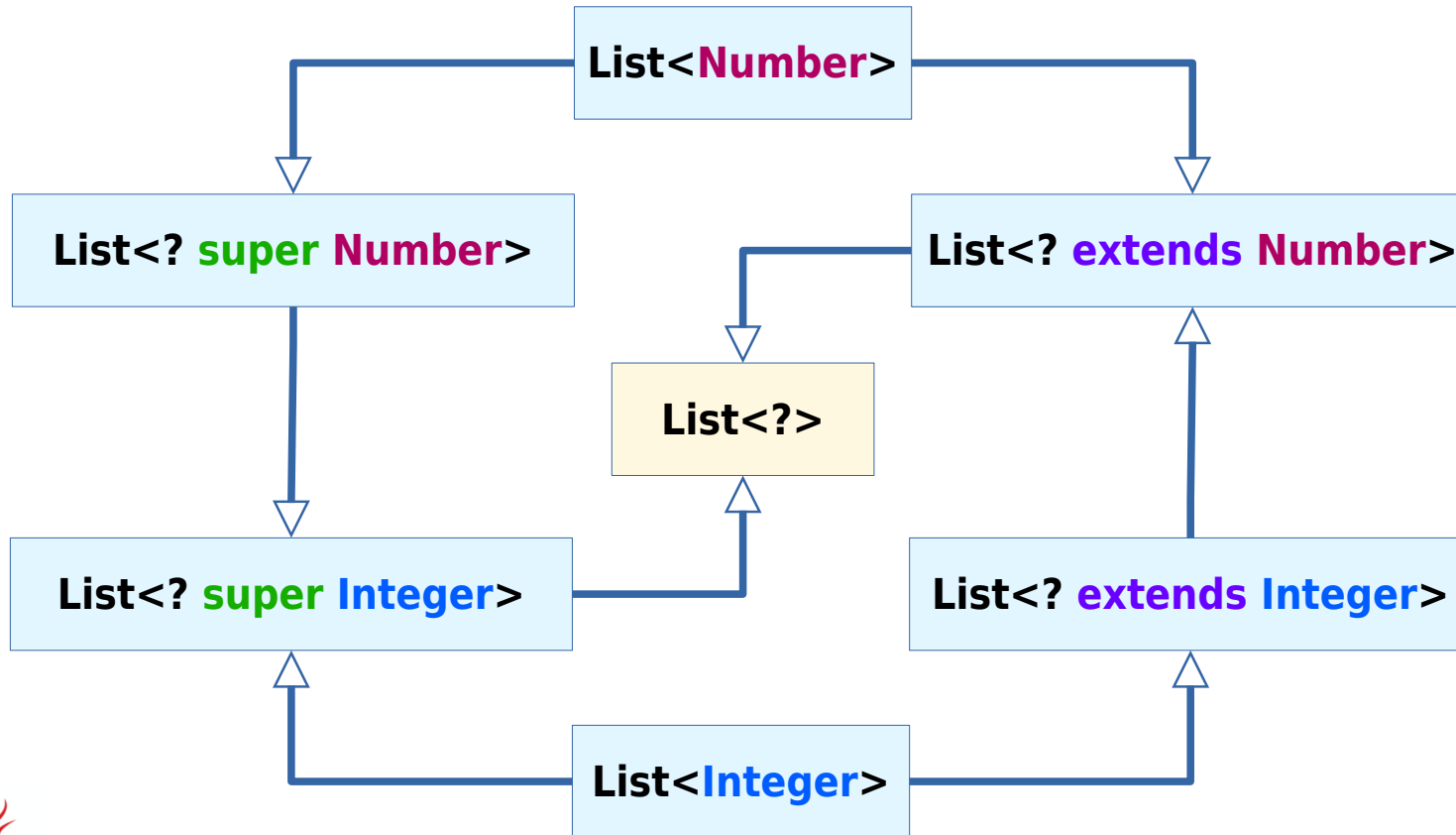
```
Box<Number> nbox = new Box<>();  
Box<Integer> ibox = new Box<>(); ibox.put(42);  
Box<Object> obox = new Box<>();  
nbox.copyFrom(ibox); // T = Number, ? = Integer  
nbox.copyTo(obox);    // T = Number, ? = Object  
ibox.copyTo(nbox);
```


- Параметры методов
 - ❖ производители (producers)
 - ❖ потребители (consumers)
- Правило PECS - Producer Extends, Consumer Super
- Производители — ограничение сверху `<? extends X>`
- `<? extends Object> == <?>`
- Потребители — ограничение снизу `<? super X>`
- Параметр одновременно потребитель и производитель — подстановки не используются (тип задается явно)

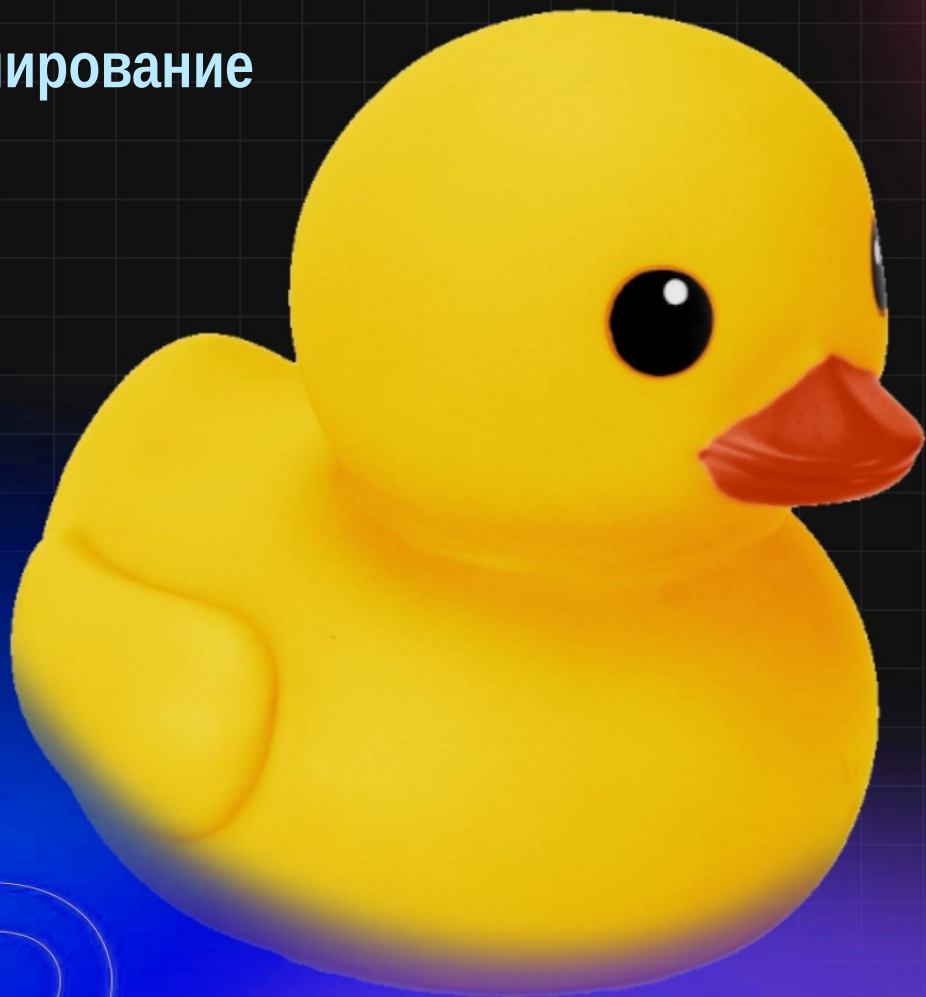
- `class List<T>`
 - ❖ `public double sum(List<? extends Number> list)`
 - ❖ `public void print(List<?> list)`
 - ❖ `public <T> void fill(List<? super T> list, T value)`
- Class `java.lang.Enum<E extends Enum<E>>`

- `class List<T>`
 - ❖ `public double sum(List<? extends Number> list)`
 - ❖ `public void print(List<?> list)`
 - ❖ `public <T> void fill(List<? super T> list, T value)`
- `Class java.lang.Enum<E extends Enum<E>>`
 - ❖ `enum Color ~> class Color extends Enum<Color>`
 - ❖ Enum - это обобщенный класс, параметром которого является класс, который может наследоваться только от Enum с таким же параметром.

- Integer - потомок Number
- Наследование массивов - ковариантно
 - ❖ Integer[] - потомок Number[]
- Наследование обобщений - инвариантно
 - ❖ Box<Integer> - не потомок и не предок Box<Number>
- Ограничение сверху - ковариантно
 - ❖ Box<? extends Integer> - потомок Box<? extends Number>
- Ограничение снизу - контрвариантно
 - ❖ Box<? super Integer> - предок Box<? super Number>



Программирование
2 семестр
2026

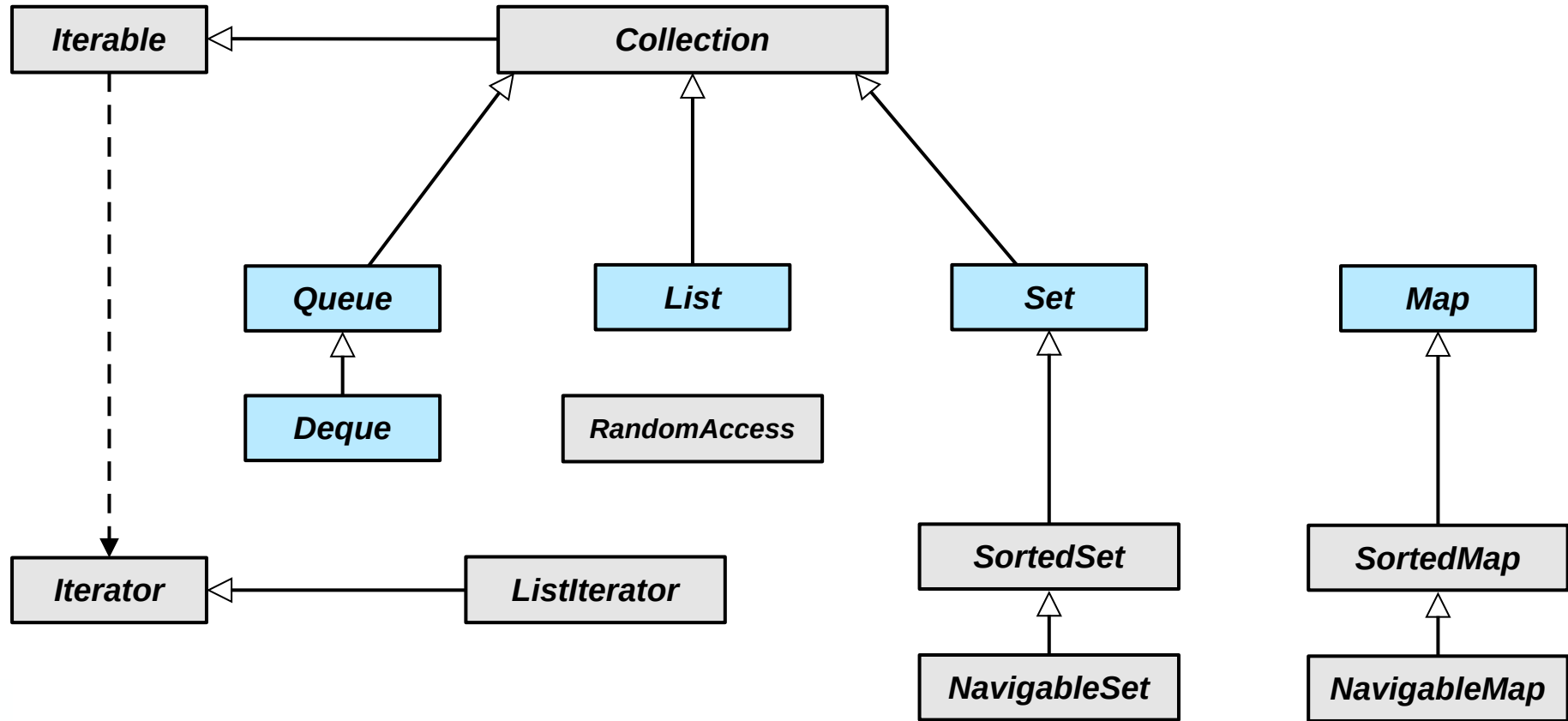


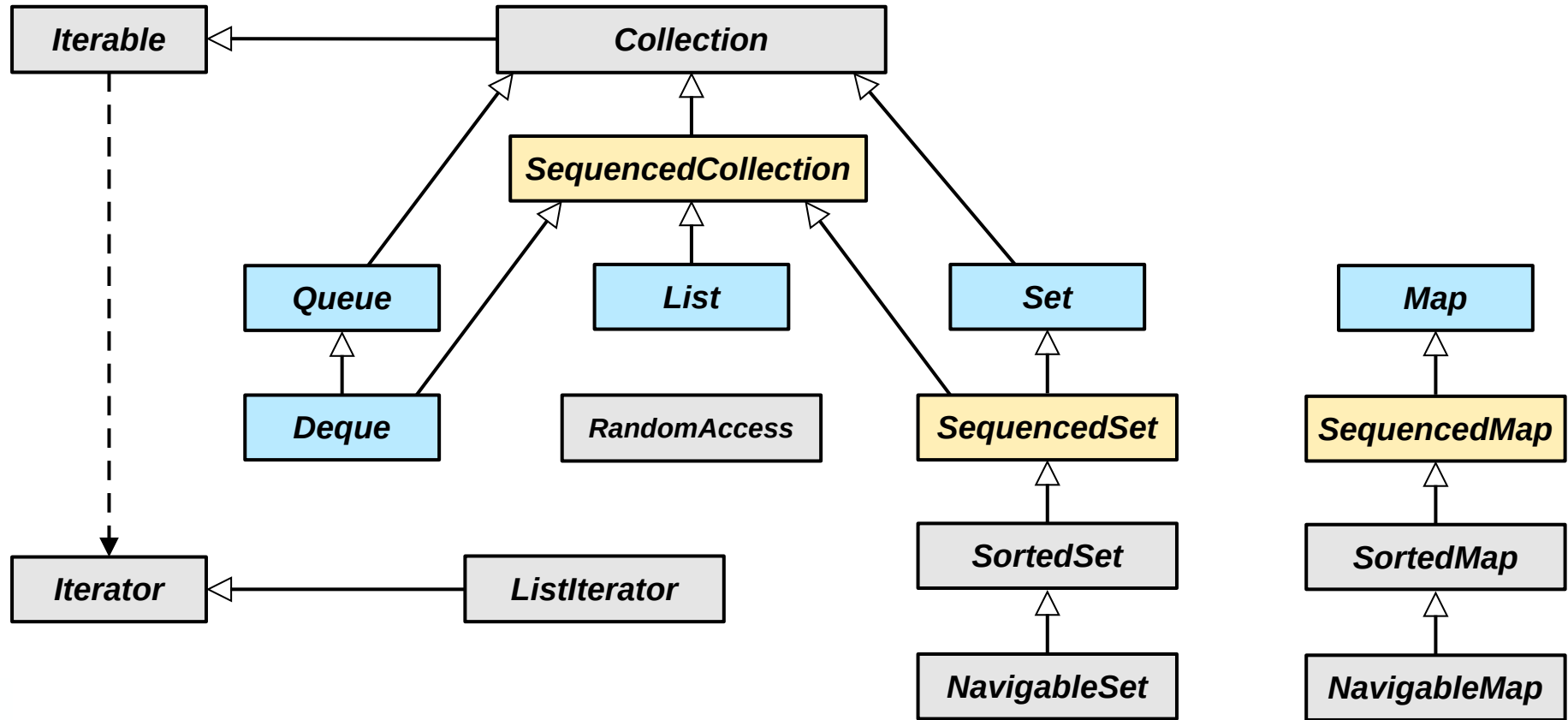
ІТМО

Коллекции

- Низкоуровневый тип
- Элементы индексированы (начиная с 0)
- Поддерживаются примитивные типы
- Размер нельзя изменить
- Быстрый доступ к любому элементу по индексу
 - ❖ $A_i = A_0 + i * \text{size}$
- Массив - Object (есть toString(), equals(), hashCode())

- Интерфейсы коллекций
- Абстрактные коллекции
- Реализации общего назначения
- Специальные реализации
- Потокобезопасные реализации
- Блокирующие очереди
- Вспомогательные классы





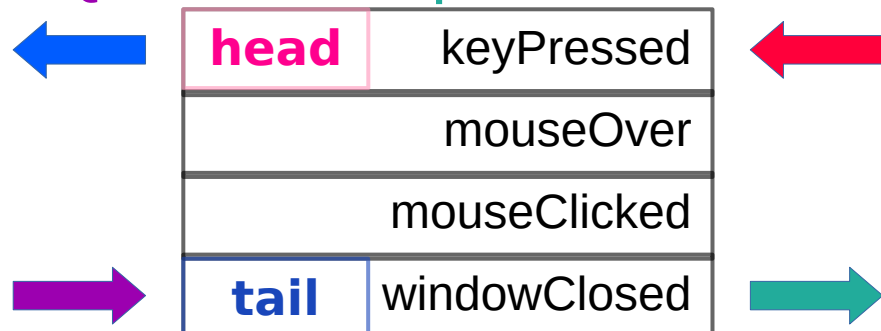
- Collection - коллекция элементов
- SequencedCollection - упорядоченная коллекция
- Map - отображение, пары "ключ-значение"
- Set - множество уникальных элементов
 - ❖ SequencedMap / Set - упорядоченное отображение / множество
 - ❖ SortedMap / Set - отсортированное отображение / множество
 - ❖ NavigableMap / Set - "навигационное" отображение / множество
- List - индексированный список
- Queue - очередь
- Deque - двусторонняя очередь

- Queue - КЬЮ - очередь
- Dequeue - ДЕК - Double-Ended QUEUE - двусторонняя очередь
- dequeue - декью - удалить из очереди
- enqueue - энкью - поставить в очередь

• List

0	яблоко
1	апельсин
2	банан
3	киви

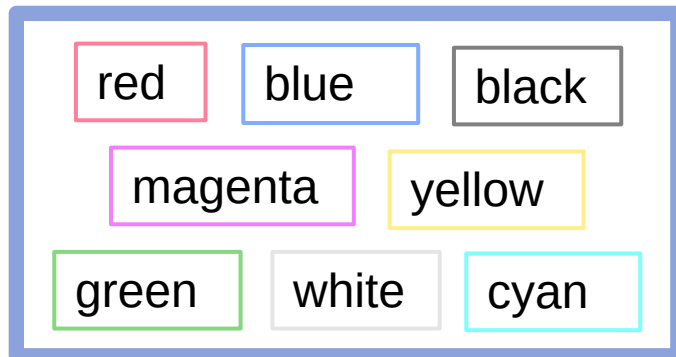
• Queue / Deque / Stack



• Map

и	37153142
т	31620970
м	16203060
о	55414481

• Set



- List : ArrayList vs LinkedList
- Set : HashSet vs LinkedHashSet vs TreeSet
- Map : HashMap vs LinkedHashMap vs TreeMap
- Queue : PriorityQueue vs LinkedList vs ArrayDeque
- Big-O нотация - асимптотическая оценка эффективности

- ❖ $O(1)$
- ❖ $O(\log n)$
- ❖ $O(n)$

- ❖ $O(n \log n)$
- ❖ $O(n^2)$
- ❖ $O(2^n)$

- Базовые интерфейсы определяют контракт
 - ❖ Что и как можно делать с элементами
 - ❖ Queue, Deque, List, Set, SortedSet, Map, SortedMap, ...
- Базовые реализации определяют характеристики
 - ❖ Скорость, занимаемая память, упорядоченность
 - ❖ Array, Linked List, Hash Table, Tree, Heap, ...
 - ❖ Big O-нотация

- `boolean add (E e)`
 - ❖ результат - **элемент есть** в коллекции
 - ❖ `true` - коллекция изменилась
- `boolean remove (Object o)`
 - ❖ результат - **элемента нет** в коллекции
 - ❖ `true` - коллекция изменилась
- `boolean contains (Object o)`
 - ❖ `true` - элемент есть, `false` - элемента нет

- `boolean add (E e)`
 - ❖ результат - элемент есть в коллекции
 - ❖ `true` - коллекция изменилась
- `boolean remove (Object o)`
 - ❖ результат - элемента нет в коллекции
 - ❖ `true` - коллекция изменилась
- `boolean contains (Object o)`
 - ❖ `true` - элемент есть, `false` - элемента нет

Внимание : тип параметра!

- `addAll(Collection<?> c)`
- `removeAll(Collection<?> c)`
- `retainAll(Collection<?> c)`
- `containsAll(Collection<?> c)`

- `clear()`
- `isEmpty()`
- `size()`

- `addAll(Collection<?> c)`
- `removeAll(Collection<?> c)`
- `retainAll(Collection<?> c)`
- `containsAll(Collection<?> c)`

- `clear()`
- `isEmpty()`
- `size()`

Внимание : тип параметра!

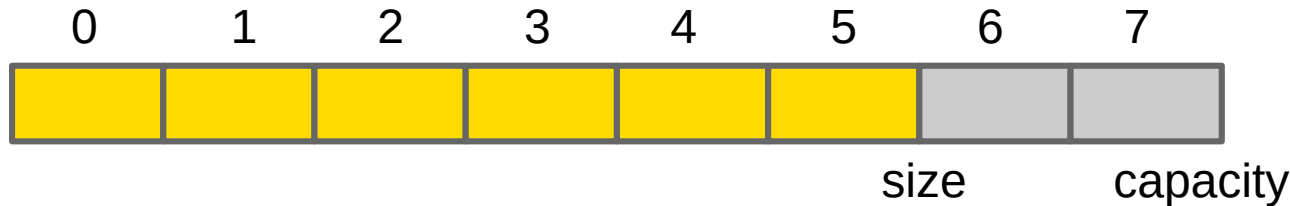
- Первый (First) и последний (Last) элементы
 - ❖ `void addFirst(E e), E getFirst(), E removeFirst()`
 - ❖ `void addLast(E e), E getLast(), E removeLast()`
 - ❖ `SequencedCollection reversed()`

- Индекс от 0 до N
- E get(int index)
- E set(int index, E e)
- void add(int index, E e)
- E remove(int index)
 - ❖ List<Integer> list
 - ❖ list.remove(1) ???

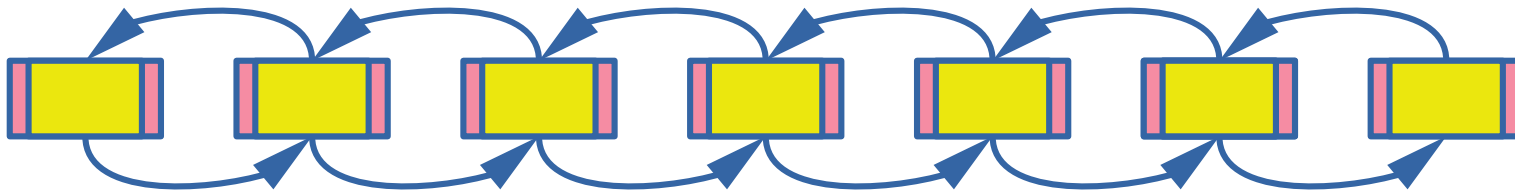
- Пример

```
List<String> list = new ArrayList<>();  
list.add("First element, index 0");  
list.add(list.get(0));  
var list2 = List<String>.of("X", "Y");  
list.add(list2);  
for (String s : list) {  
    System.out.println(s);  
}  
list.clear();
```

- Реализация списка (list) на основе массива (array)
- ArrayList extends AbstractList
implements List, RandomAccess
- Быстрый произвольный доступ по индексу - $O(1)$



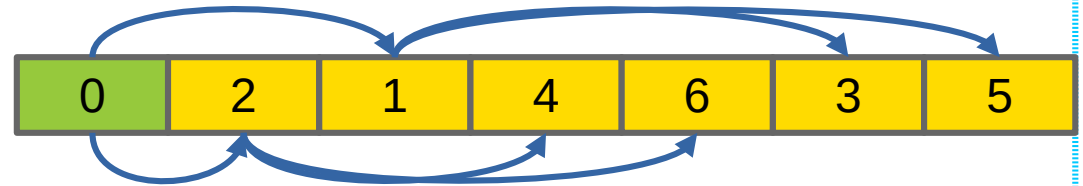
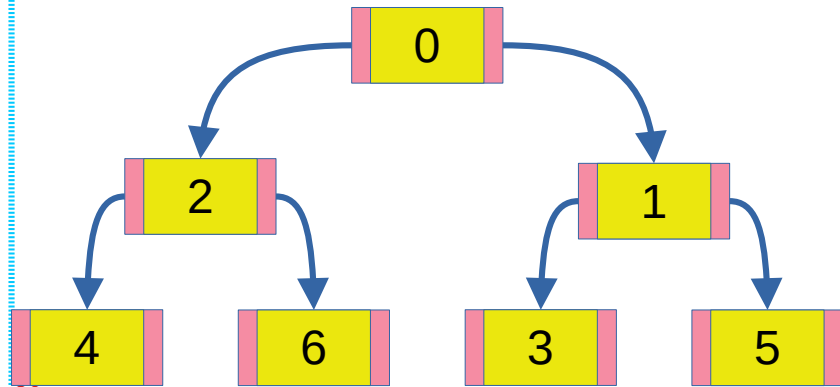
- Реализация на основе двусвязного списка
- LinkedList extends AbstractSequentialList implements List
- Последовательный доступ к элементам
- Быстрое добавление и удаление - $O(1)$



- Vector
 - ❖ потокобезопасный, но устаревший ArrayList
- Stack
 - ❖ устаревшая реализация на базе вектора
- CopyOnWriteArrayList
 - ❖ новая копия при каждом изменении

- Как правило - **FIFO (first in, first out)**
- Добавляем в конец, забираем из начала
- Методы, бросающие исключения
 - ❖ `boolean add()` , `E remove()`, `E element()`
- Методы, возвращающие особые значения (`null`)
 - ❖ `boolean offer()`, `E poll()`, `E peek()`

- Реализация на основе кучи (heap) - дерево или массив
- В начале очереди - наименьший элемент, дальше - ?
- FIFO + очередь тех, кто без очереди
- Нужна реализация сравнения элементов



- Двусторонняя очередь
- Может быть стеком
- Методы
 - ❖ Исключения
 - addFirst, removeFirst, getFirst
 - addLast, removeLast, getLast
 - ❖ null
 - offerFirst, pollFirst, peekFirst
 - offerLast, pollLast, peekLast

- Пример

```
Queue<String> queue = new  
ArrayDeque<>();  
queue.add("First element, index 0");  
queue.add("One more");  
var list = List<String>.of("X", "Y");  
queue.add(list);  
while (!queue.empty()) {  
    System.out.println(queue.remove());  
}
```

• Queue

- ❖ add(e)
- ❖ offer(e)
- ❖ remove()
- ❖ poll()
- ❖ element()
- ❖ peek()

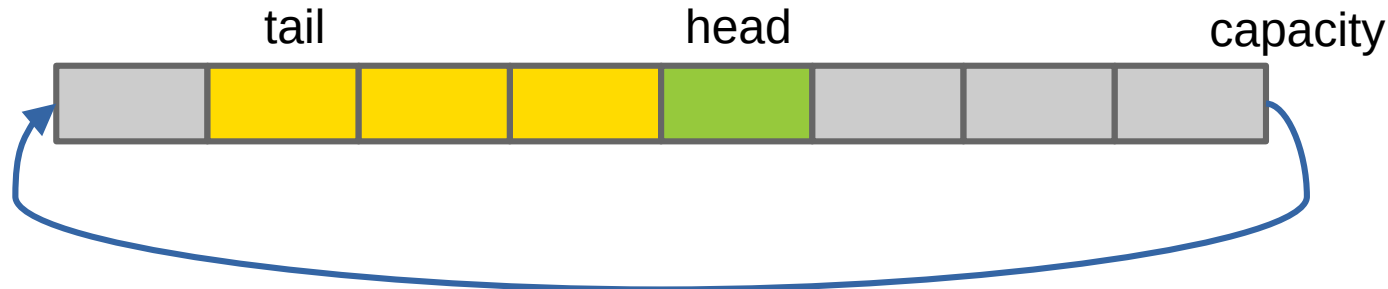
• Deque

- ❖ addFirst(e)
- ❖ addLast(e)
- ❖ offerLast(e)
- ❖ removeFirst()
- ❖ pollFirst()
- ❖ getFirst()
- ❖ peekFirst()

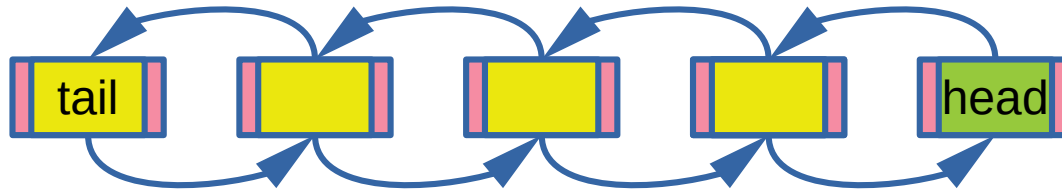
• Stack

- ❖ push(e)
- ❖ pop()
- ❖ peek()

- Реализация на основе массива
- Быстрое добавление и удаление - $O(1)$



- LinkedList implements Queue, Deque, List
- Быстрые добавление, удаление - $O(1)$,



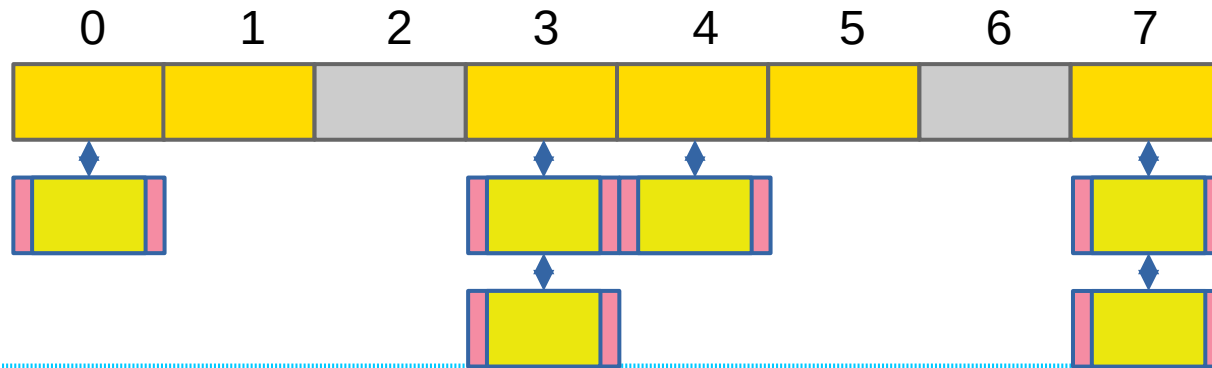
- Ассоциативный массив, отображение, словарь
- Два параметра типа
- Ключи уникальные, значения - любые
- Методы:
 - ❖ V put(K key, V value)
 - ❖ V get(Object K)
 - ❖ V remove(Object k)
- Представления:
 - ❖ keySet(), values(), entrySet()

- Пример

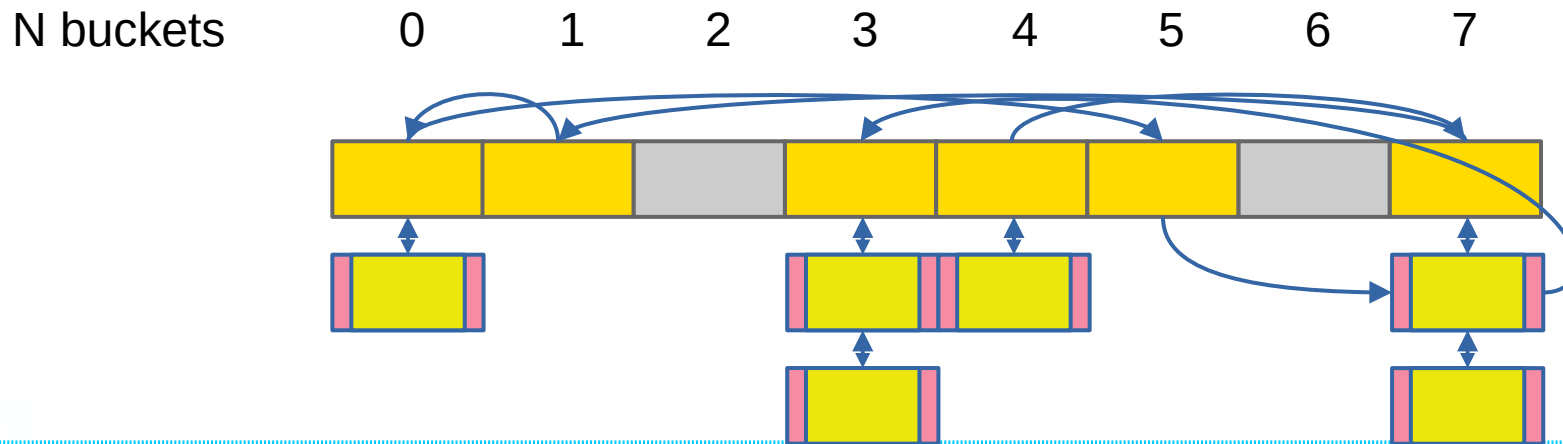
```
var map = new HashMap<String, Byte>();  
map.put("red", 60);  
map.put("green", 120);  
for (int i : map.values()) {  
    System.out.println(i);  
}
```

- Ассоциативный массив на основе хеш-таблицы
- $\text{bucket} = \text{key.hashCode()} \% N$
- Нужен метод `hashCode()` у элементов
- Обход элементов - в полном беспорядке

N buckets



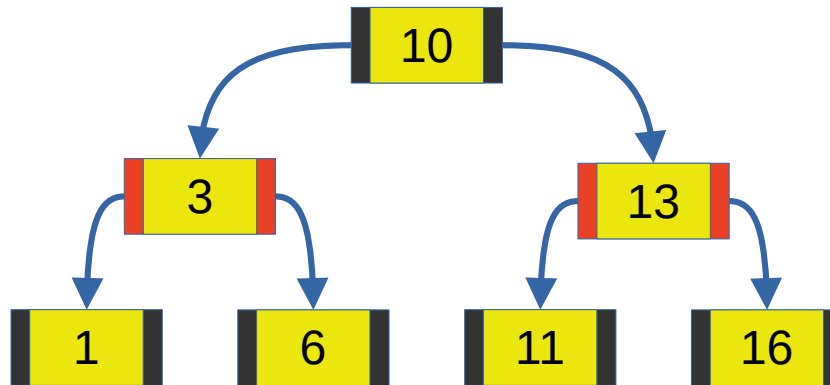
- Упорядоченность
- HashMap + связный список
- Получение элементов в порядке добавления



- SortedMap<K, V> extends SequencedMap
 - ❖ K firstKey(), K lastKey()
 - ❖ SortedMap subMap(K from, K to)
 - ❖ SortedMap headMap(K k), SortedMap tailMap(K k)
- NavigableMap<K,V> extends SequencedMap, SortedMap
 - ❖ lowerKey, floorKey, ceilingKey, higherKey (<, <=, >= >)
 - ❖ lowerEntry, floorEntry, ceilingEntry, higherEntry
 - ❖ subMap, headMap, tailMap - (K key, boolean inclusive)

- Sequenced $\approx$ Ordered - упорядоченный (есть порядок)
 - ❖ первый (first) элемент, последний (last) элемент
 - ❖ порядок добавления элементов
 - ❖ Indexed - индексированный - пронумерованный
- Sorted - отсортированный (по какому-то критерию)
 - ❖ первый - например, наименьший
- Navigable - можно перейти к большему или меньшему
 - ❖ floor / lower / higher / ceiling

- Реализация на основе Red-Black Tree
- Самобалансирующееся бинарное дерево
- Элементы отсортированы
- Нужна реализация сортировки



- Hashtable
 - ❖ устаревшая синхронизированная реализация
- WeakHashMap
 - ❖ ключи - слабые ссылки (кэш)
- EnumMap
 - ❖ ключи - enum, реализация на основе массива
- IdentityHashMap
 - ❖ сравнение ключей по ==, а не по equals()

- Методы - как у Collection
- Массовые операции - объединение, пересечение, разность множеств
- Реализации - как у Map
 - ❖ HashSet implements Set,
 - ❖ LinkedHashSet extends HashSet implements SequencedSet,
 - ❖ TreeSet implements NavigableSet (SequencedSet, SortedSet)
 - ❖ Set реализован на основе Map

- CopyOnWriteArraySet
 - ❖ новая копия при изменении
- EnumSet
 - ❖ реализация на основе битовой карты

- List / Set

- ❖ .of()
- ❖ .of(E e1)
- ❖ .of(E e1, E e2)
- ❖ ... (до 10 элементов)
- ❖ .of(E... elements)
- ❖ .copyOf(Collection c)

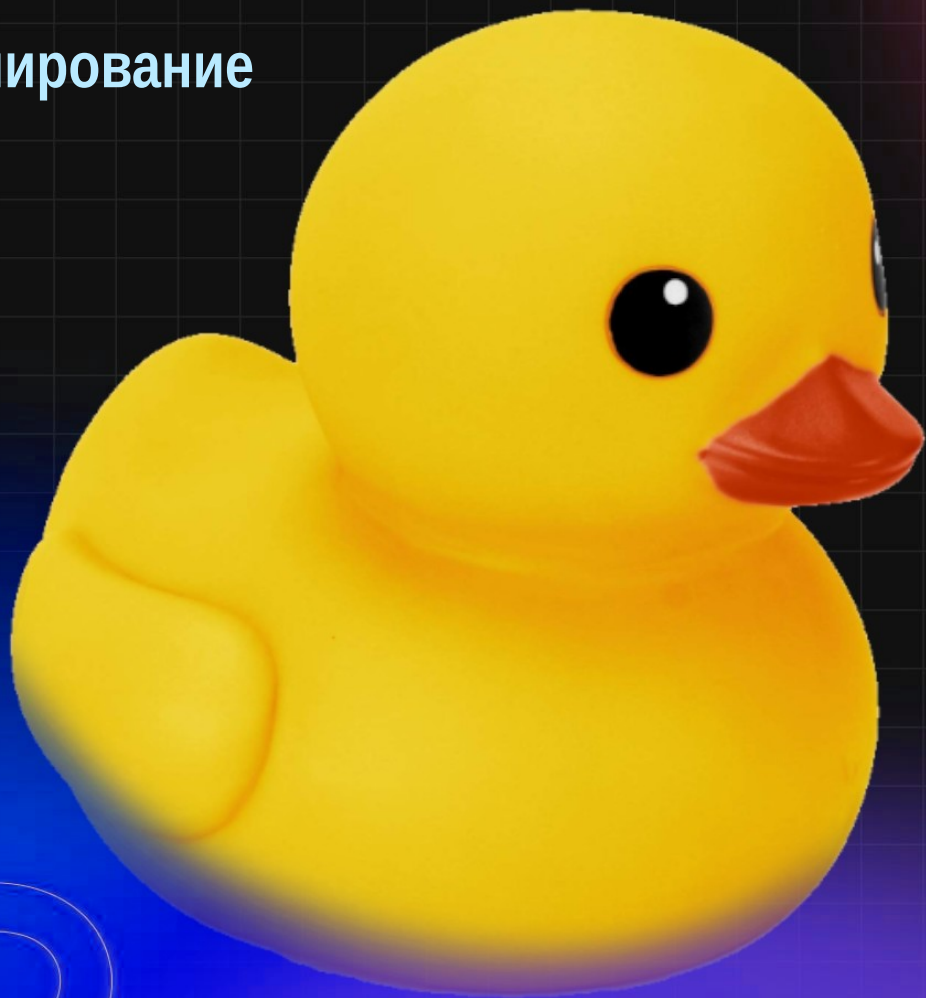
- Map

- ❖ .of()
- ❖ .of(K k1, V v1)
- ❖ .of(K k1, V v1, K k2, V v2)
- ❖ ... (до 10 элементов)
- ❖ .ofEntries(Map.Entry... entries)
 - Map.Entry entry(K k, V v)
- ❖ .copyOf(Map m)

- коллекции-обертки (представления):
 - ❖ synchronized
 - ❖ unmodifiable
 - ❖ checked
- алгоритмы для List
 - ❖ sort(), shuffle(), reverse(), fill(), swap(), binarySearch()
- фабрики коллекций
 - ❖ emptySet, emptyList, emptyMap,
 - ❖ singleton, singletonList, singletonMap
- и еще много полезного (Queue asLifoQueue(Deque))

- Методы для работы с массивами (обычными)
 - ❖ сортировка, поиск, копирование, заполнение
 - ❖ и еще много полезного
 - ❖ `public static <T> List<T> asList(T... a)`
 - возвращает список на основе массива a
 - элементы менять можно
 - размер менять нельзя

Программирование
2 семестр
2026



ІТМО

Шаблоны
проектирования
(введение)

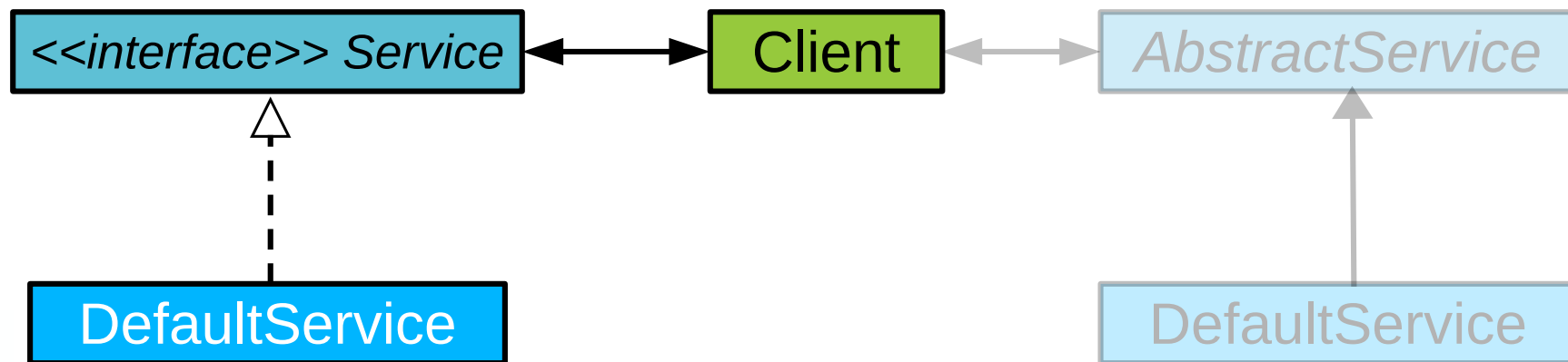
- Кто-то когда-то уже делал что-то похожее
- Пришлось вносить изменения - возникли проблемы
- Нужно сразу было делать по-другому!

- GoF Book
 - ❖ Gang of Four



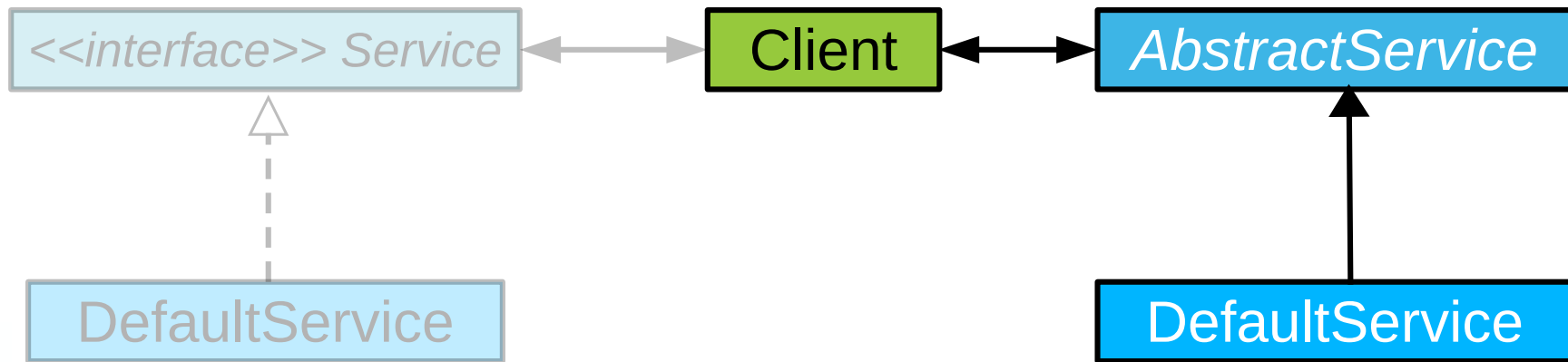
- Независимость от реализации - больше гибкость и универсальность

- Унификация поведения потомков - проще реализация

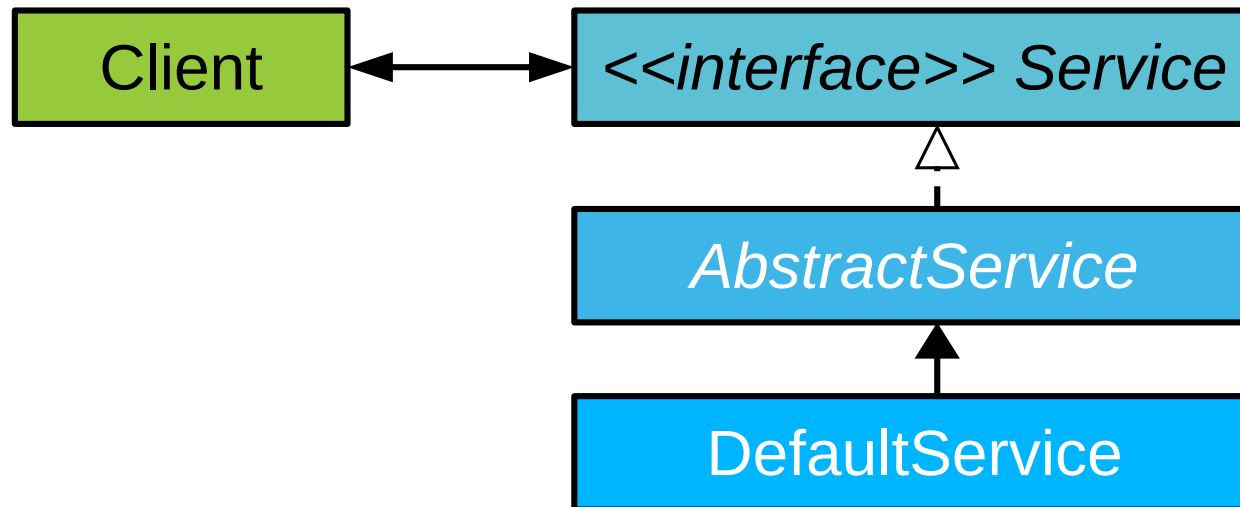


- Независимость от реализации - больше гибкость и универсальность

- Унификация поведения потомков - проще реализация



- Общие свойства - АБСТРАКТНЫЙ КЛАСС
- Взаимодействие с клиентом - ИНТЕРФЕЙС
- $\text{Set}\langle T \rangle \leftarrow \text{AbstractSet}\langle T \rangle \leftarrow \text{HashSet}\langle T \rangle$

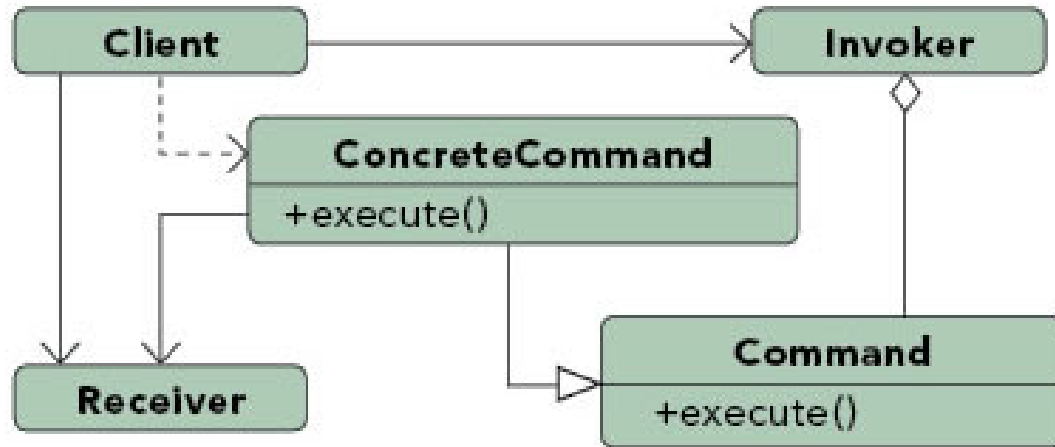


- Внимательно прочитать задание и подумать
 - ❖ Посмотреть любой вариант лаб 6, 7 и 8
- Выделить классы (не забывать SOLID)
 - ❖ Подумать над общей структурой
 - ❖ паттерн Command
- Нарисовать диаграмму
 - ❖ Объяснить уточке, как это должно работать
- Написать маленький работающий код
 - ❖ Дописывать до победы


```
Scanner sc = new Scanner(System.in);
String line = sc.next();
String[] tokens = line.split(" ");
if (tokens[0].equals("help") {
    doHelp();
}
if (tokens[0].equals("add") {
    doAdd();
}
...

public void doHelp() {
    System.out.println("help - помощь");
    System.out.println("add - добавить элемент");
}
```

- Управление командами
- Разделение вызова и исполнения команд
- Можно организовать очередь команд и макрокоманды



```
public interface Command {           // Abstract Command
    void execute();
}
public class CollectionManager {      // Receiver (исполнитель)
    List<Pokemon> pokemonList = new ArrayList<>();
    public add(Pokemon p) {
        pokemonList.add(p);
    }
}
public class AddCommand implements Command { // Add Command
    CollectionManager cm;
    public void execute() {
        cm.add(pokemon);
    }
}
```

```
public class Invoker {  
    ...  
    Map<String, Command> commands = new HashMap<>();  
    commands.put("help", new HelpCommand());  
    commands.put("add", new AddCommand());  
    ...  
    Scanner sc = new Scanner(System.in);  
    while (sc.hasNext()) {  
        String line = sc.next();  
        String[] tokens = line.split(" ");  
        Command command = commands.get(tokens[0]);  
        command.execute();  
    }  
}
```

```
public interface Command {                // Abstract Command
    void execute();
    String descr();
}

public class HelpCommand implements Command {
    public String descr() { return "help - помощь"; }
    public void execute() {
        for (Command c : commands.values()) {
            System.out.println(c.descr());
        }
    }
}
```